AI projects(generative and agentic AI)

1.TripMate AI: A Multi-Agent Travel Planner with LangGraph

- **Overview:** An open-source AI travel planner. It turns a natural-language trip request into flight suggestions, hotel ideas, and a day-by-day itinerary.
- **Problem it solves:** Planning a trip usually means switching between many websites, tools, and spreadsheets. This brings the whole flow into one experience.
- **Agents:** A flight-search agent, a hotel-research agent, an itinerary-planning agent, and a final response agent, all coordinated by a LangGraph workflow.
- **Features:**
    - Flight research with AviationStack
    - Hotel suggestions using Tavily search
    - Multi-agent orchestration with LangGraph
    - Structured itinerary generation
    - FastAPI backend with a simple web interface
    - Conversation state saved in PostgreSQL
    - Fast LLM responses with Groq
- **Tech stack:** Python 3.10+, FastAPI, Jinja2, HTML/CSS/JavaScript, LangGraph, LangChain, Groq LLMs, PostgreSQL, Tavily API, AviationStack API
- **How it works:** The user submits a request, then the flight, hotel, itinerary, and final agents run in that order.

2 AI Meeting Assistant: Transcribe, Summarize, and Chat with Any Meeting

- **Overview:** An AI tool that turns a meeting or video link into a text transcript and a full summary. You can then ask questions about the content.
- **Problem it solves:** Long meetings and videos take a lot of time to rewatch, and key points get missed. This tool gives you the transcript and summary, and you can ask about anything you didn't follow.
- **Features:**
    - Transcribes audio from YouTube, Zoom, or other meeting links, and from uploaded files
    - Works with a locally hosted Whisper model, so there is no paid transcription API
    - Supports both English and Hindi transcription
    - Generates a structured summary with action items
    - Translates Hindi to English for multilingual audio
    - Question-and-answer chat over the transcript, using RAG
    - Exports the results as PDF or TXT
    - Deployed on Render
- **Tech stack:** Python, LangChain (LCEL), OpenAI Whisper, ChromaDB, HuggingFace embeddings, Mistral API, Streamlit
- **How it works:** The user provides a link or file, Whisper transcribes the audio, the LLM summarizes it, and the transcript is stored as embeddings in ChromaDB so the user can ask questions about it.

3.Multi-Agent Research Assistant: Deep Research on Any Topic

- **Overview:** A multi-agent AI system that does deep research on any topic and returns complete, structured information in real time.
- **Problem it solves:** Researching a topic by hand means searching many sources, reading long pages, and writing up the findings. This system does the whole process automatically and gives you a finished report.

- **Agents:** Four specialized LLM agents work together through a shared-state pipeline:
  - **Search agent:** finds relevant, up-to-date sources on the web
  - **Reader agent:** extracts the key content from those sources
  - **Writer agent:** turns the findings into a structured report
  - **Critic agent:** reviews and scores the report for quality
- **Features:**
  - Live web research using the Tavily Search API
  - Autonomous collaboration between agents, with no manual steps
  - Scored and reviewed final report
  - Interactive web interface built with Streamlit
  - Fast responses from a Groq-hosted LLM
- **Tech stack:** Python, LangChain, Groq LLM, Tavily API, Streamlit, BeautifulSoup
- **How it works:** The user enters a topic. The search agent gathers sources, the reader agent extracts the details, the writer agent drafts the report, and the critic agent reviews and scores it.

4.VTU Mysore Assistant: A Domain-Based RAG Chatbot for Campus Queries

- **Overview:** An AI assistant for VTU Mysore. It answers student and applicant questions about the four programmes offered: B.Tech, MCA, MBA, and M.Tech.
- **Problem it solves:** Applicants and students have to search through brochures, notices, and websites for details like fees and admission steps. This assistant gives them accurate answers in one place.
- **Features:**
  - Domain selection: the user picks B.Tech, MCA, MBA, or M.Tech, and answers are tailored to that programme
  - A RAG system built on a curated knowledge base that covers fees and the admission process
  - Document embeddings for semantic retrieval, so answers come from the actual data and not guesses
  - General question support, so users can also ask questions outside the stored data

Tech stack: Python, LangChain (LCEL), ChromaDB, HuggingFace embeddings, Mistral API, Streamlit

How it works: The user selects a domain and asks a question. The system retrieves the most relevant information from that domain's embedded data and generates an answer grounded in it.

5.Document Assistant AI: A RAG-Based PDF Question Answering System

- Overview: An AI-powered document assistant that allows users to upload PDF documents, ask questions about their content, and generate short summaries. It uses Retrieval-Augmented Generation (RAG) to retrieve relevant information from documents and generate context-based answers.

- Problem it solves: Reading lengthy PDF documents and manually searching for specific information can be time-consuming. This project makes it easier to understand documents, find relevant information, and get quick summaries in one place.

- Core components: A document-processing component for extracting text and creating chunks, a vector database for storing document embeddings, a retrieval component for finding relevant

information, and an LLM-powered response component for answering questions and summarizing documents.

- Features:

- PDF document upload and text extraction
- Text chunking for efficient document processing
- Vector embeddings and storage in a vector database
- Semantic search to retrieve relevant document content
- Question answering based on the uploaded PDF
- Automatic short summaries of PDF documents
- RAG pipeline for context-aware responses
- LangGraph integration for workflow orchestration
- Interactive user interface built with Streamlit
- Hugging Face integration for AI models

- Tech stack: Python, Streamlit, Retrieval-Augmented Generation (RAG), Vector Database, Hugging Face, LangGraph, Large Language Models (LLMs).

- How it works: The user uploads a PDF document, the system extracts its text and splits it into smaller chunks. The chunks are converted into vector embeddings and stored in a vector database. When the user asks a question, the system retrieves the most relevant chunks and uses them as context to generate an answer. The system can also generate a short summary of the document to help the user understand its main points quickly.

PostPilot: A Self-Reviewing LinkedIn Post Generator

- **Overview:** An AI app that writes LinkedIn posts from a topic or short summary. One LLM drafts the post and a second LLM reviews it, and the two repeat this loop until the post meets the required quality.
- **Problem it solves:** Writing a good LinkedIn post takes time, and a single AI draft often sounds generic or misses the intended message. This project adds a built-in reviewer that critiques and improves each draft, so the final post is polished without manual editing.
- **Agents / Components:**
  - Generator LLM that writes the LinkedIn post from the user's topic or summary
  - Reviewer LLM that evaluates the draft for clarity, tone, structure, and how well it matches the required summary
  - Feedback loop that sends the reviewer's suggestions back to the generator for a revised draft
  - Stopping condition that ends the cycle once the reviewer approves the post or a maximum number of iterations is reached
  - User interface for entering the topic and viewing the final post
- **Features:**
  - Two-LLM generator and reviewer architecture
  - Iterative refinement loop that repeats until the post is satisfactory
  - Structured reviewer feedback with specific improvement points
  - Control over post tone, length, and style
  - Iteration limit to avoid endless loops and control API cost
  - Final post ready to copy and publish on LinkedIn
- **Tech stack:** Python, LangChain, LLM APIs (such as Groq or OpenAI), Streamlit
- **How it works:** The user enters a topic or summary, then the generator LLM writes the first draft. The reviewer LLM checks it against the required summary and gives feedback. If the draft isn't approved, the feedback goes back to the generator for a revision, and this repeats until the reviewer is satisfied. The approved post is then shown to the user.

-

PulseAgent: A Multi-Agent Real-Time News and Weather Assistant

- **Overview:** An AI assistant built with LangChain that answers questions about the latest news and current weather. A human approval step lets the user review and confirm the output before the final response is delivered.
- **Problem it solves:** Checking news and weather means jumping between different apps and websites, and fully automated AI systems can act on wrong or unwanted requests. This project brings both into one conversation and keeps the user in control of the workflow.
- **Agents / Components:**
  - Router agent that understands the request and decides whether it needs news, weather, or both
  - News agent that fetches real-time headlines using a LangChain tool and summarizes them
  - Weather agent that fetches live weather data for the requested location using a LangChain tool
  - Human-in-the-loop checkpoint where the user approves, edits, or rejects before the workflow continues
  - Response agent that combines the results into one clear answer
  - LangChain chains and agent executors that connect the agents and pass data between them

- **Features:**
  - Real-time news retrieval and summarization
  - Live weather updates for any city
  - Multi-agent coordination using LangChain agents, tools, and chains
  - Human-in-the-loop approval for greater control and reliability
  - Conversation memory maintained across turns
  - Clean, structured final responses
- **Tech stack:** Python, LangChain, LLM API (such as Groq), News API, Weather API, Streamlit
- **How it works:** The user submits a request, then the router agent decides which agents to call. The news and weather agents fetch live data through their tools, and the flow pauses at the human checkpoint for the user to approve or modify the result. Once approved, the response agent presents the final news and weather update.

MACHINE LEARNING AND DATA SCIENCE PROJECTS

CineMatch: A Content-Based Movie Recommendation System

- **Overview:** A movie recommendation app that suggests similar movies based on a title the user selects. It shows the recommendations with posters and details in a simple web interface.
- **Problem it solves:** With thousands of movies available, choosing what to watch takes time and often ends in endless scrolling. This project gives quick, relevant suggestions based on a movie the user already likes.
- **Models / Components:**
  - Data cleaning and preprocessing pipeline
  - TF-IDF vectorizer to convert movie text into numerical features
  - Cosine similarity to find the closest matching movies
  - TMDB API integration to fetch posters and movie details
  - Streamlit interface for user interaction
- **Features:**
  - Cleaned movie dataset with missing values and duplicates handled
  - Text-based feature extraction from genres, overview, keywords, and cast
  - Content-based recommendations using TF-IDF and similarity scores
  - Live movie posters and details fetched using the TMDB API
  - Simple, interactive Streamlit web app
  - Fast recommendations through precomputed similarity data
- **Tech stack:** Python, Pandas, NumPy, Scikit-learn (TF-IDF, cosine similarity), TMDB API, Streamlit
- **How it works:** The user selects a movie in the app, then the system compares its TF-IDF vector against all other movies using cosine similarity. The top matches are picked, and their posters and details are fetched from TMDB and displayed.

EmoSense: A Text-Based Emotion Prediction App

- **Overview:** A machine learning app that reads a user's comment and predicts the emotion behind it. Results appear instantly in a simple web interface.
- **Problem it solves:** Comments, reviews, and social media posts are hard to analyze by hand, and the emotion behind the text is easy to miss. This project automates emotion detection so feedback can be understood quickly and consistently.
- **Models / Components:**
  - Text cleaning and preprocessing pipeline
  - Feature engineering module to convert text into numerical features
  - Multiple ML classifiers trained and compared (for example, Logistic Regression, Naive Bayes, SVM, Random Forest)
  - Best-performing model selected and saved for prediction
  - Streamlit interface for real-time predictions
- **Features:**
  - Cleaned dataset of comments labeled with emotions
  - Text preprocessing such as lowercasing, removing punctuation and stopwords, and tokenization
  - Feature engineering to turn comments into model-ready input
  - Training and comparison of several ML algorithms
  - Evaluation using metrics like accuracy, precision, recall, and F1-score
  - Real-time emotion prediction from any comment typed into the app
  - Simple, interactive Streamlit web app
- **Tech stack:** Python, Pandas, NumPy, Scikit-learn, NLTK, Streamlit
- **How it works:** The user types a comment into the app, then the text is cleaned and converted into features using the same pipeline used in training. The saved model predicts the emotion, and the result is displayed on screen.

MindCheck: A Mental Health Prediction App

- **Overview:** A machine learning app that takes a person's survey or lifestyle responses and predicts their likely mental health condition or risk level. Results appear in a simple web interface.
- **Problem it solves:** Mental health issues often go unnoticed because people hesitate to seek help or don't recognize the early signs. This project offers a quick, private first screening that can encourage people to seek professional support sooner.
- **Models / Components:**
  - Data cleaning and preprocessing pipeline
  - Feature engineering and encoding module for categorical and numerical inputs
  - Multiple ML classifiers trained and compared (for example, Logistic Regression, Random Forest, SVM, XGBoost)
  - Best-performing model selected and saved for prediction
  - Streamlit interface for user input and results
- **Features:**
  - Cleaned dataset with missing values, outliers, and inconsistent entries handled
  - Encoding of categorical responses and scaling of numerical features
  - Training and comparison of several ML algorithms
  - Evaluation using accuracy, precision, recall, F1-score, and confusion matrix
  - Real-time prediction from a user-filled form
  - Simple, interactive Streamlit web app

    - Clear note that results are a screening aid, not a medical diagnosis
- **Tech stack:** Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Streamlit
- **How it works:** The user fills in the form in the app, then the inputs are encoded and scaled using the same pipeline used in training. The saved model predicts the mental health outcome, and the result is displayed on screen.

CardioGuard: A Heart Disease Prediction App

- **Overview:** A machine learning app that takes a patient's health details, such as age, blood pressure, cholesterol, and heart rate, and predicts whether they are at risk of heart disease. Results appear instantly in a simple web interface.
- **Problem it solves:** Heart disease is a leading cause of death, and early warning signs are often missed until it is too late. This project gives a fast, data-driven risk check that can prompt people to get timely medical attention.
- **Models / Components:**
    - Data cleaning and preprocessing pipeline
    - Feature engineering and scaling module for clinical attributes
    - Multiple ML classifiers trained and compared (for example, Logistic Regression, KNN, Random Forest, SVM)
    - Best-performing model selected and saved for prediction
    - Streamlit interface for entering patient details
- **Features:**
    - Cleaned clinical dataset with missing values and outliers handled
    - Exploratory data analysis of factors like age, cholesterol, chest pain type, and blood pressure
    - Feature scaling and encoding for model-ready input
    - Training and comparison of several ML algorithms
    - Evaluation using accuracy, precision, recall, F1-score, and ROC-AUC
    - Real-time risk prediction from patient details entered in the app
    - Simple, interactive Streamlit web app
- **Tech stack:** Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Streamlit
- **How it works:** The user enters health details into the app, then the inputs are scaled and encoded using the same pipeline used in training. The saved model predicts whether there is a risk of heart disease, and the result is displayed on screen.